Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 Set

02

03

04

05

Set

- Set is another data structure supported by Python
- Basically, sets are same as lists but with the difference that- sets are lists with no duplicate entry
- Set is mutable

Creating a Set

- A set is created by placing all the elements inside {}
- Elements are separated by comma

```
S = {1, 6, 2, 6, 2, 0, 5, 4}
print(S)
```

Creating a Set

```
S = {1, ‘a’, 2.3, “abc”}
print(S)
```

Creating a Set

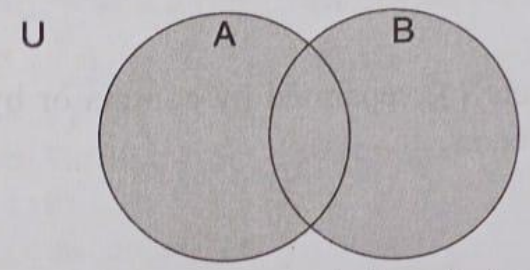
```
#Converting a list into a Set
List1 = [1, 3, 2]
S = set(List1)
print(S)
```

Creating a Set

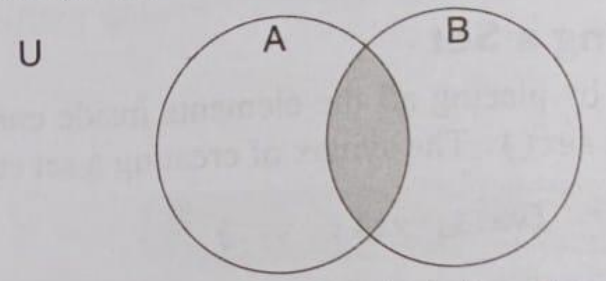
```
#Converting a tuple into a Set
Tup = (1, 3, 2)
S = set(Tup)
print(S)
```

Set Operations

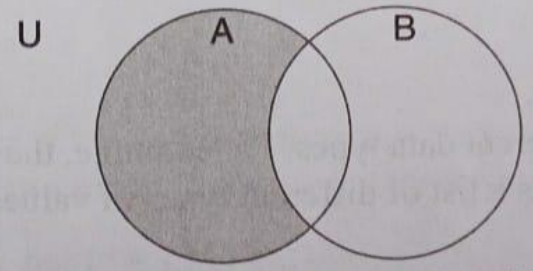
Like in case of mathematical sets, Python sets are also a powerful tool as they have the ability to calculate differences and intersections between other sets. Figures 8.5(a–d) given below demonstrate various set operations.



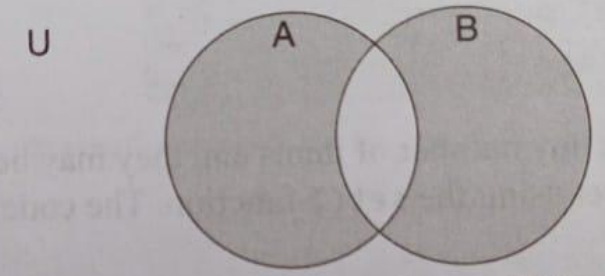
Union of two sets includes all elements from both the sets



Intersection of two sets includes elements that are common to both the sets



Difference of A and B (A–B) includes all elements that are in A but not in B



Symmetric difference of A and B includes all elements that are in A and B but not the one that are common to A and B

Figure 8.5 Set operations

Set Operations

- Like mathematical sets, Python sets are also very powerful
- They have the ability to calculate differences and intersections etc.

```
S1 = {1, 2, 3}
S2 = {3, 4, 5}
S3 = S1.intersection(S2)
print(S3)
{3}
```

Set Operations

```
S1 = {1, 2, 3}
S2 = {3, 4, 5}
S3 = S1.union(S2)
print(S3)
{1, 2, 3, 4, 5}
```

Set Operations

```
S1 = {1, 2, 3}
S2 = {3, 4, 5}
S3 = S1.difference(S2)
print(S3)
{1, 2}

S4 = S2.difference(S1)
print(S4)
{4, 5}
```

Set Operations

```
# Symmetric Difference: all elements except the common elements
S1={1,2,3}
S2={3,4,5}
S3=S1.symmetric_difference(S2)
print(S3)
{1, 2, 4, 5}
```

Set Operations

Operation	Description	Code	Output

`s.update(t)`	Adds elements of set t in the set s provided that all duplicates are avoided	`s = set([1,2,3,4,5])`
		`t = set([6,7,8])`
		`s.update(t)`
		`print(s)`
			`(1,2,3,4, 5,6,7,8)`

`s.add(x)`	Adds element x to the set s provided that all duplicates are avoided	`s = set([1,2,3,4,5])`
		`s.add(6)`
		`print(s)`
			`(1,2,3,4, 5,6)`

`s.remove(x)`	Removes element x from set s. Returns KeyError if x is not present	`s = set([1,2,3,4,5])`
		`s.remove(3)`
		`print(s)`
			`(1,2,4,5)`

`s.discard(x)`	Same as `remove()` but does not give an error if x is not present in the set	`s = set([1,2,3,4,5])`
		`s.discard(3)`
		`print(s)`
			`(1,2,4,5)`

`s.pop()`	Removes and returns any arbitrary element from s. KeyError is raised if s is empty	`s = set([1,2,3,4,5])`
		`s.pop()`
		`print(s)`
			`(2,3,4,5)`

`s.clear()`	Removes all elements from the set	`s = set([1,2,3,4,5])`
		`s.clear()`
		`print(s)`
			`set()`

Set Operations

`len(s)`	Returns the length of set	`s = set([1,2,3,4,5])` `print(len(s))`	`5`
`x in s`	Returns True is x is present in set s and False otherwise	`s = set([1,2,3,4,5])` `print(3 in s)`	`True`
`x not in s`	Returns True is x is not present in set s and False otherwise	`s = set([1,2,3,4,5])` `print(6 not in s)`	`True`

`s.issubset(t)` or `s<=t`	Returns True if every element in set s is present in set t and False otherwise	`s = set([1,2,3,4,5])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`print(s<=t)`
			`True`

`s.issuperset(t)` or `s>=t`	Returns True if every element in t is present in set s and False otherwise	`s = set([1,2,3,4,5])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`print(s.issuperset(t))`
			`False`

`s.union(t)` or `s|t`	Returns a set s that has elements from both sets s and t	`s = set([1,2,3,4,5])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`print(s|t)`
			`(1,2,3,4, 5,6,7,8, 9,10)`

`s.intersection(t)` or `s&t`	Returns a new set that has elements which are common to both the sets s and t	`s = set([1,2,3,4,5])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`z = s&t`
		`print(z)`
			`(1,2,3, 4,5)`

`s.intersection_ update(t)`	Returns a set that has elements which are common to both the sets s and t	`s = set([1,2,10,12])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`s.intersection_update(t)`
		`print(s)`
			`(1,2,10)`

Set Operations

`s.difference(t)` or `s-t`	Returns a new set that has elements in set s but not in t	`s = set([1,2,10,12])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`z = s-t`
		`print(z)`
			(12)

`s.difference_ update(t)`	Removes all elements of another set from this set	`s = set([1,2,10,12])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`s.difference_update(t)`
		`print(s)`
			(12)

`s.symmetric_ difference(t)` or `s^t`	Returns a new set with elements either in s or in t but not both	`s = set([1,2,10,12])`
		`t = set([1,2,3,4,5,6,7,8,9, 10])`
		`z = s^t`
		`print(z)`
			(3,4,5,6, 7,8,9,12)

Contd.

Set Operations

`s.copy()`	Returns a copy of set s	`s = set([1,2,10,12])`
		`t = set([1,2,3,4,5,6,7,8,9,10])`
		`print(s.copy())`
			`(1,2,12,10)`

`s.isisjoint(t)`	Returns True if two sets have a null intersection	`s = set([1,2, 3])`
		`t = set([4,5,6])`
		`print(s.isdisjoint(t))`
			`True`

`all(s)`	Returns True if all elements in the set are True and False otherwise	`s = set([0,1,2,3,4])` `print(all(s))`	`False`
`any(s)`	Returns True if any of the elements in the set is True. Returns False if the set is empty	`s = set([0,1,2,3,4])` `print(any(s))`	`True`

Set Operations

`enumerate(s)`	Returns an enumerate object which contains index as well as value of all the items of set as a pair	`s = set(['a','b','c','d'])`
		`for i in enumerate(s):`
		`  print(i,end= ' ')`
			`(0, 'a') (1, 'c') (2, 'b') (3, 'd')`

`max(s)`	Returns the maximum value in a set	`s = set([0,1,2,3,4,5])` `print(max(s))`	`5`
`min(s)`	Returns the minimum value in a set	`s = set([0,1,2,3,4,5])` `print(min(s))`	`0`
`sum(s)`	Returns the sum of elements in the set	`s = set([0,1,2,3,4,5])` `print(sum(s))`	`15`
`sorted(s)`	Return a new sorted list from elements in the set. It does not sorts the set as sets are immutable.	`s = set([5,4,3,2,1,0])` `print(sorted(s))`	`[0,1,2,3, 4,5]`

`s == t and s != t`	s == t returns True if the two set are equivalent and False otherwise. s!=t returns True if both sets are not equivalent and False otherwise	`s = set(['a','b','c'])`
		`t = set("abc")`
		`z = set(tuple('abc'))`
		`print(s == t)`
		`print(s!=z)`
			`True`
			`False`

Key Points to Remember

- Set can’t contain other mutable objects like list
- Sets don’t allow users to access or change an element using indexing or slicing

Thank You

Dr. Abdul Kayom


Asst. Prof, (SENSE)


kayomabdul@vitap.ac.in


8474860025